

Lecture 15

PyTorch Continued & Backpropagation

Backpropagation – computing gradients easily and efficiently!

EECS 189/289, Fall 2025 @ UC Berkeley

Joseph E. Gonzalez and Narges Norouzi

Do not edit
How to change the
design



**Join at slido.com
#3940975**

① The Slido app must be installed on every computer you're presenting from

slido



Implement Gradient Descent with Optimizer

In PyTorch you implement **your own gradient descent loop** but you can use built in update rules (e.g., Adam) called **optimizers**.

```
loader = DataLoader(tr_data, batch_size=32, shuffle=True)
optimizer = torch.optim.Adam(model.parameters(), eta)
```

Minibatch SGD
DataLoader

Adam optimizer

```
for epoch in range(nepochs):
```

Epoch is one pass over data

```
    for (x, t) in loader:
```

Load one minibatch at a time

```
        optimizer.zero_grad()
```

```
        loss = loss_fn(model(x), t)
```

Compute the gradient
of the loss

```
        loss.backward()
```

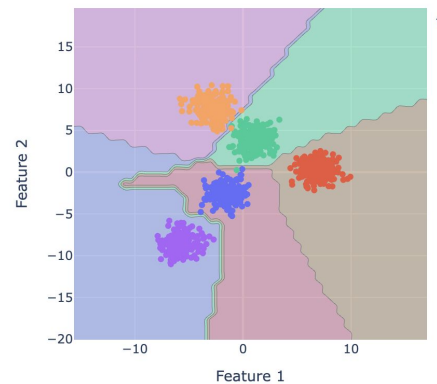
```
        optimizer.step()
```

Use optimizer to update all the parameters (1-step)

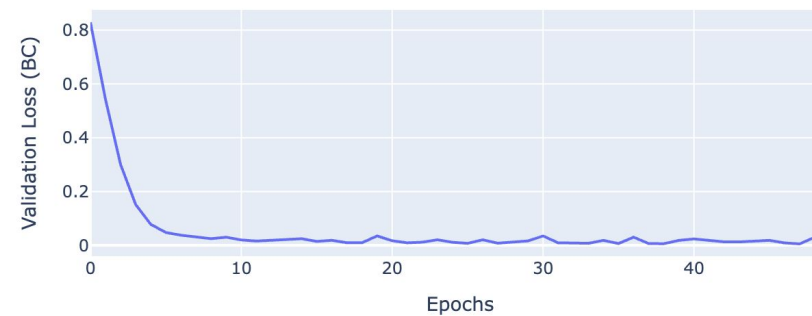
Demo



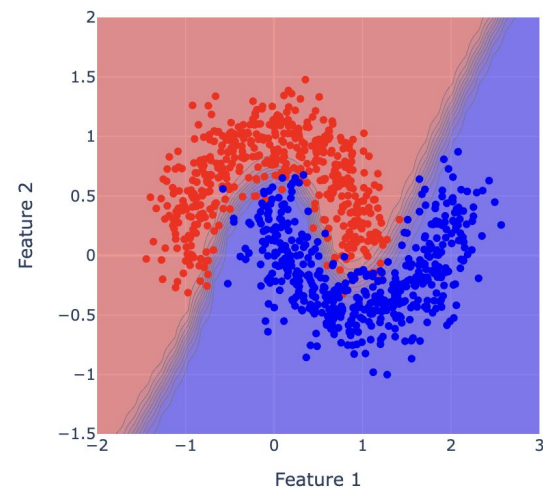
3940975



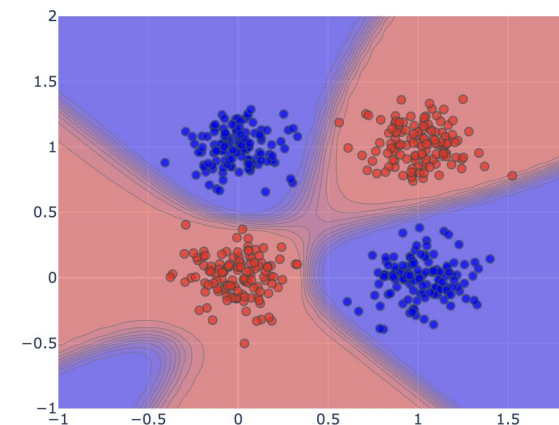
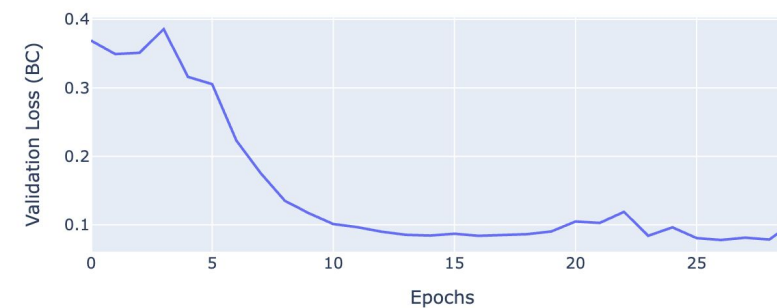
Validation Loss



make_circles Dataset



Validation Loss





Recap: Stochastic Gradient Descent

Iteratively refine the weights by **computing the gradient** of the loss on a small sample of data.

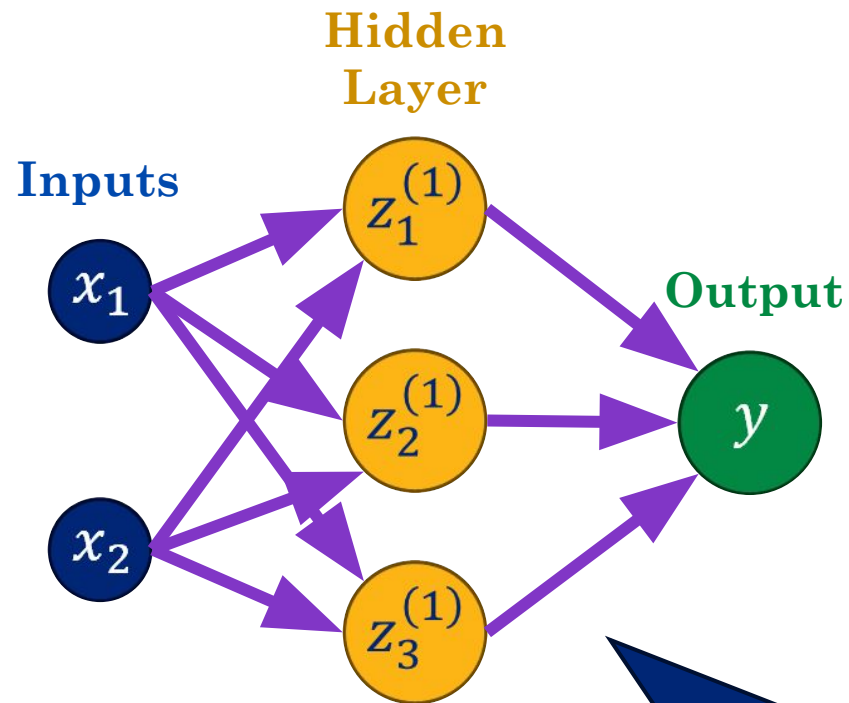
```
w(0) = Initialize()  
for  $\tau$  in range(1,  $\tau_{\max}$ ):  
     $\mathcal{B}$  = RandBatch(0,  $N$ , size =  $B$ )  
     $w^{(\tau)} = w^{(\tau-1)} + \text{Step}(\nabla E_{\mathcal{B}}(w^{(\tau-1)}), \tau, \beta, \eta)$   
    if Converged( $w^{(\tau)}, w^{(\tau-1)}, \epsilon$ ): terminate
```

- This algorithm is central to all recent advances in AI
- Today we will learn how to compute $\nabla E_{\mathcal{B}}$ **efficiently** and **automatically**



Simple Running Example

Consider the following single hidden layer classification network



No bias at each layer in this example.

$$z_i^{(1)} = \sigma \left(\sum_{j=1}^2 w_{ij}^{(1)} x_j \right)$$

Sigmoid Activation Function

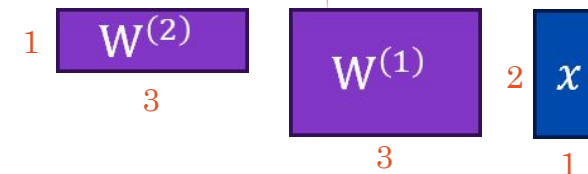
$$\sigma(t) = \frac{1}{1 + e^{-t}}$$

$$y(\mathbf{x}, \mathbf{w}) = \sigma \left(\sum_{i=1}^3 w_{1i}^{(2)} z_i^{(1)} \right) = \sigma \left(\sum_{i=1}^3 w_{1i}^{(2)} \sigma \left(\sum_{j=1}^2 w_{ij}^{(1)} x_j \right) \right)$$

Substituting $z_i^{(1)}$

$$p(t = 1 | \mathbf{x}, \mathbf{w}) = y(\mathbf{x}, \mathbf{w}) = \sigma \left(\mathbf{W}^{(2)} \sigma(\mathbf{W}^{(1)} \mathbf{x}) \right)$$

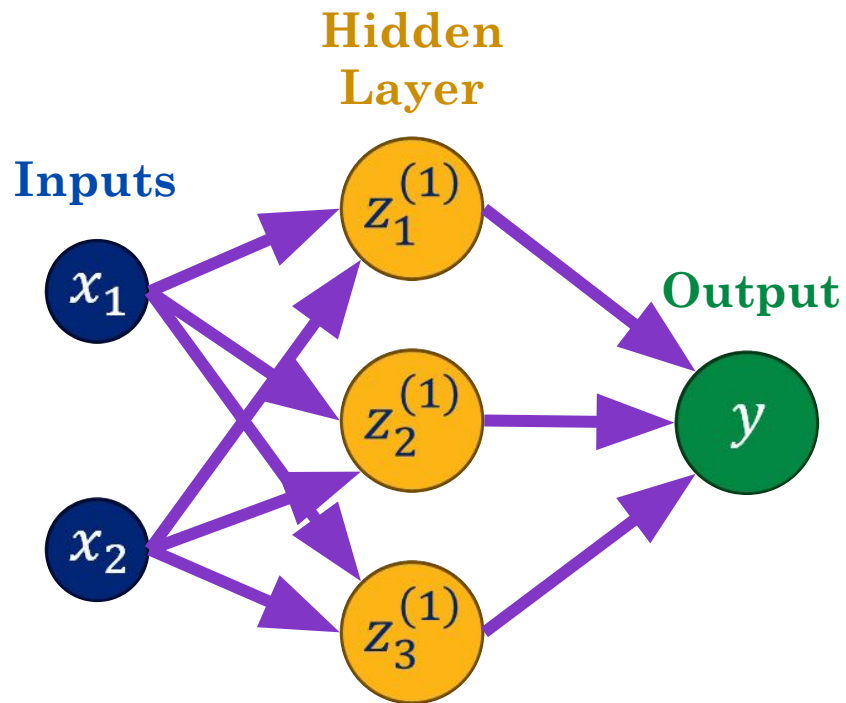
Matrix Notation





Defining the Error Function

Computing the negative log likelihood of the data $\mathcal{D}$:



Sigmoid Activation Function

$$\sigma(t) = \frac{1}{1 + e^{-t}}$$

$$p(t = 1 \mid \mathbf{x}, \mathbf{w}) = y(\mathbf{x}, \mathbf{w}) = \sigma\left(W^{(2)} \sigma(W^{(1)} \mathbf{x})\right)$$

$$E(\mathbf{w}; \mathcal{D}) = -\ln \left(\prod_{d=1}^D y(\mathbf{x}_d, \mathbf{w})^{t_d} (1 - y(\mathbf{x}_d, \mathbf{w}))^{(1-t_d)} \right)$$

Likelihood of the data.

$$E(\mathbf{w}; \mathcal{D}) = - \sum_{d=1}^D t_d \ln(y(\mathbf{x}, \mathbf{w})) + (1 - t_d) \ln(1 - y(\mathbf{x}, \mathbf{w}))$$

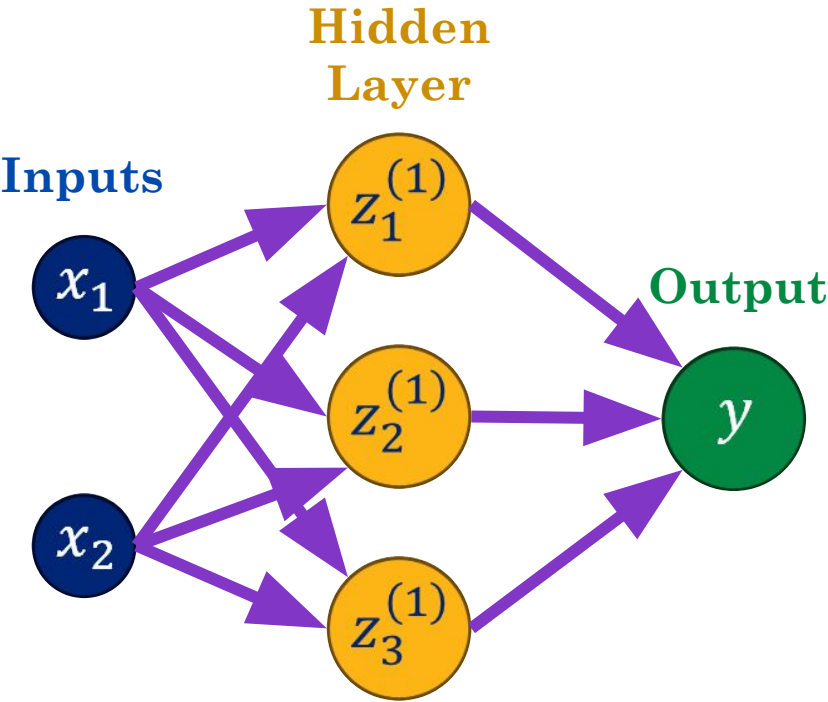
Algebra!



Computing the Gradient

Computing the gradient ∇_w of the negative log likelihood with respect to w :

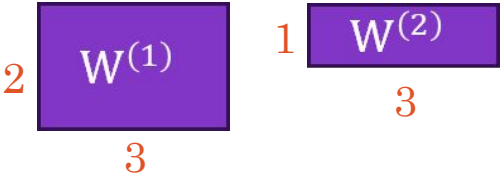
$$p(t = 1 \mid x, w) = y(x, w) = \sigma \left(W^{(2)} \sigma(W^{(1)} x) \right)$$
$$E(w; \mathcal{D}) = - \sum_{d=1}^D t_d \ln(y(x, w)) + (1 - t_d) \ln(1 - y(x, w))$$



Sigmoid Activation Function

$$\sigma(t) = \frac{1}{1 + e^{-t}}$$

What is the shape of the gradient?
Recall $w = \{W^{(1)}, W^{(2)}\}$





What is the shape of the gradient?

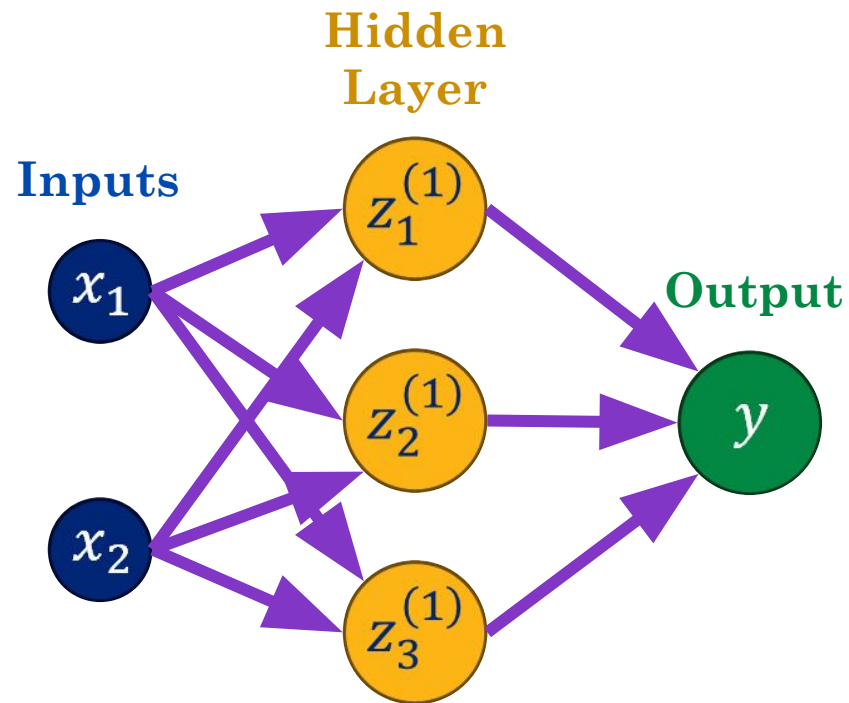


Computing the Gradient

Computing the gradient $\nabla_{\mathbf{w}}$ of the negative log likelihood with respect to $\mathbf{w}$:

$$p(t = 1 \mid \mathbf{x}, \mathbf{w}) = \mathbf{y}(\mathbf{x}, \mathbf{w}) = \sigma \left(\mathbf{W}^{(2)} \sigma(\mathbf{W}^{(1)} \mathbf{x}) \right)$$

$$E(\mathbf{w}; \mathcal{D}) = - \sum_{d=1}^D t_d \ln(\mathbf{y}(\mathbf{x}, \mathbf{w})) + (1 - t_d) \ln(1 - \mathbf{y}(\mathbf{x}, \mathbf{w}))$$

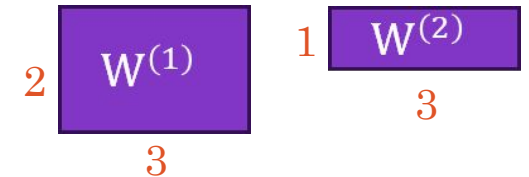


Sigmoid Activation Function

$$\sigma(t) = \frac{1}{1 + e^{-t}}$$

What is the shape of the gradient?

Recall $\mathbf{w} = \{\mathbf{W}^{(1)}, \mathbf{W}^{(2)}\}$



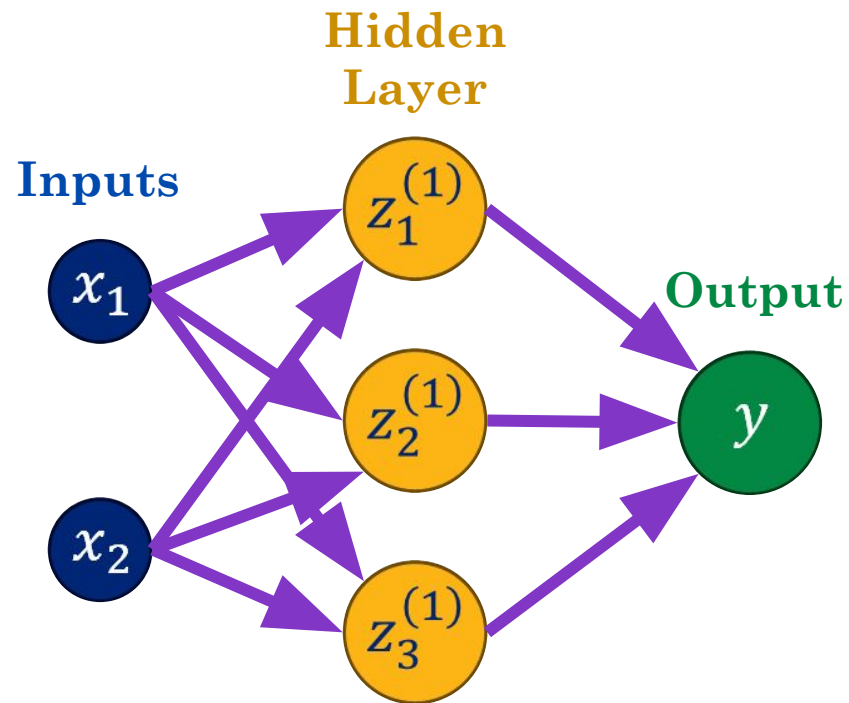
The gradient $\nabla_{\mathbf{w}}$ is the **vector** of **partial derivatives** for all the entries in $\mathbf{W}^{(1)}$ and $\mathbf{W}^{(2)}$.

Here $\nabla_{\mathbf{w}} E(\mathbf{w}; \mathcal{D}) \in \mathbb{R}^{(2 \times 3 + 3)} = \mathbb{R}^9$ (9 edge in network graph)



Computing the Gradient

Using the chain rule to compute the partial derivative with respect to w_i



Sigmoid Activation Function

$$\sigma(t) = \frac{1}{1 + e^{-t}}$$

$$p(t = 1 \mid \mathbf{x}, \mathbf{w}) = \mathbf{y}(\mathbf{x}, \mathbf{w}) = \sigma\left(\mathbf{W}^{(2)} \sigma(\mathbf{W}^{(1)} \mathbf{x})\right)$$

$$E(\mathbf{w}; \mathcal{D}) = - \sum_{d=1}^D t_d \ln(\mathbf{y}(\mathbf{x}, \mathbf{w})) + (1 - t_d) \ln(1 - \mathbf{y}(\mathbf{x}, \mathbf{w}))$$

Recall that $\frac{\partial}{\partial q} \ln(q) = \frac{1}{q}$ then with the **chain rule**:

$$\frac{\partial}{\partial w_i} E(\mathbf{w}; \mathcal{D}) = - \sum_{d=1}^D t_d \left(\frac{1}{\mathbf{y}(\mathbf{x}, \mathbf{w})} \right) \frac{\partial \mathbf{y}(\mathbf{x}, \mathbf{w})}{\partial w_i} + (1 - t_d) \left(\frac{1}{1 - \mathbf{y}(\mathbf{x}, \mathbf{w})} \right) \frac{\partial \mathbf{y}(\mathbf{x}, \mathbf{w})}{\partial w_i}$$

We need to compute the **partial derivative of the network** $\mathbf{y}(\mathbf{x}, \mathbf{w})$ with respect to **each weight** w_i

Numerical Differentiation



Proposal: Numerical Differentiation

We could estimate the gradient numerically by computing:

$$\frac{\partial}{\partial w_i} E(\mathbf{w}; \mathcal{D}) \approx \frac{E(\mathbf{w} + \epsilon I_i; \mathcal{D}) - E(\mathbf{w} - \epsilon I_i; \mathcal{D})}{2\epsilon}$$

for a small value of ϵ (where I_i is the i^{th} column of the identity matrix).

```
def numeric_gradient(f, x1, x2, h=1e-8):  
    """  
    Compute the numerical derivative of f with respect to x1 at (x1, x2)  
    using central difference.  
    """  
    return [(f(x1 + h, x2) - f(x1 - h, x2)) / (2 * h),  
            (f(x1, x2 + h) - f(x1, x2 - h)) / (2 * h)]
```

✓ 0.0s

```
numeric_gradient(f, 1, 2)
```

✓ 0.0s

```
[np.float64(16.778112232884723), np.float64(8.80520287793729)]
```



Proposal: Numerical Differentiation

Advantage?

- Can apply to ***any* error function** and network architecture.
- Easy to implement.

Disadvantage?

- Approximate – but likely to be **fairly accurate** for small ϵ .
- Expensive! – Why?



Cost of Numerical Differentiation

Computing the **gradient** using numerical differentiation:

$$\frac{\partial}{\partial w_i} E(\mathbf{w}; \mathcal{D}) \approx \frac{E(\mathbf{w} + \epsilon \mathbf{I}_i; \mathcal{D}) + E(\mathbf{w} - \epsilon \mathbf{I}_i; \mathcal{D})}{2\epsilon}$$

for a D dimensional weight vector $\mathbf{w} \in \mathbb{R}^D$ requires:

$$\left[\frac{\partial}{\partial w_1} E(\mathbf{w}; \mathcal{D}), \dots, \frac{\partial}{\partial w_D} E(\mathbf{w}; \mathcal{D}) \right]$$

$2D$ error calculations. Recall each **error calculation** requires $\mathcal{O}(ND)$ computation for N datapoints.

- **Total cost:** $\mathcal{O}(ND^2)$ per gradient step!

- For real problems (minibatch) often $N \approx 1000$ and $D \gg 10^6 \rightarrow ND^2 \gg 10^{3+2*6} = 10^{15}$
- Stochastic gradient with batch size 1: $D^2 \gg 10^{12}$

Symbolic Differentiation



Proposal: Symbolic Differentiation

Continue to use calculus to derive the gradient equation.

$$\frac{\partial}{\partial \mathbf{w}_i} E(\mathbf{w}; \mathcal{D}) = - \sum_{d=1}^D t_d \left(\frac{1}{y(\mathbf{x}, \mathbf{w})} \right) \frac{\partial y(\mathbf{x}, \mathbf{w})}{\partial \mathbf{w}_i} + (1 - t_d) \left(\frac{1}{1 - y(\mathbf{x}, \mathbf{w})} \right) \frac{\partial y(\mathbf{x}, \mathbf{w})}{\partial \mathbf{w}_i}$$

for our small network: $y(\mathbf{x}, \mathbf{w}) = \sigma \left(\mathbf{W}^{(2)} \sigma(\mathbf{W}^{(1)} \mathbf{x}) \right)$ and using the identity:

$$\frac{\partial}{\partial t} \sigma(t) = \sigma(t)(1 - \sigma(t))$$

Then taking the partial derivative of the ij^{th} entry of $\mathbf{W}^{(1)}$ we get:

$$\frac{\partial}{\partial W_{ij}^{(1)}} y(\mathbf{x}, \mathbf{w}) = \sigma \left(\mathbf{W}^{(2)} \sigma(\mathbf{W}^{(1)} \mathbf{x}) \right) \left(1 - \sigma \left(\mathbf{W}^{(2)} \sigma(\mathbf{W}^{(1)} \mathbf{x}) \right) \right) \mathbf{W}^{(2)} \frac{\partial}{\partial W_{ij}^{(1)}} \sigma(\mathbf{W}^{(1)} \mathbf{x})$$

$$\frac{\partial}{\partial W_{ij}^{(1)}} y(\mathbf{x}, \mathbf{w}) = y(\mathbf{x}, \mathbf{w}) (1 - y(\mathbf{x}, \mathbf{w})) W_j^{(2)} \sigma(\mathbf{W}^{(1)} \mathbf{x}) \left(1 - \sigma(\mathbf{W}^{(1)} \mathbf{x}) \right) x_i$$

Painful and error prone (there is probably a mistake on this slide?) – **Not automatic**.



Symbolic Computer Algebra Systems

Computer algebra systems like SymPy can be used to automate the differentiation process.

- Express functions **symbolically**
- Automatically construct **derivative expressions**

```
n = sp.symbols('n', integer=True)      # number of data points (symbolic length)
i = sp.Idx('i', n)                     # index variable for summation
w0, w1 = sp.symbols('w0 w1')           # weights
x = sp.IndexedBase('x', shape=(n,))    # indexed variable for x data
y = sp.IndexedBase('y', shape=(n,))    # indexed variable for y data
```

```
E_i = (y[i] - sp.sin(w0 + w1 * x[i]))**2
E = sp.summation(E_i, (i, 0, n - 1)) / n
```

```
gE = [sp.diff(E, var) for var in (w0, w1)]
gE
```

✓ 0.0s

Python

```
[Sum(-2*(-sin(w0 + w1*x[i]) + y[i])*cos(w0 + w1*x[i]), (i, 0, n - 1))/n,
Sum(-2*(-sin(w0 + w1*x[i]) + y[i])*cos(w0 + w1*x[i])*x[i], (i, 0, n - 1))/n]
```



3940975

Demo

SymPy

```
import sympy as sp
# define our symbols
x1, x2 = sp.symbols('x1 x2')
# Define a symbolic expression for the error
E = x1 * x2 + sp.exp(x1 * x2) - sp.sin(x2)
# Compute the gradient of E with respect to x1 and x2
gE = [sp.diff(E, var) for var in (x1, x2)]
gE
```



Symbolic Computer Algebra Systems

Computer algebra systems like SymPy can be used to automate the differentiation process.

- Express functions **symbolically**
- Automatically construct **derivative expressions**

Advantages:

- **Automatic:** algebraic systems can be applied to “any” expression
 - Some issues with error functions that contain control flow/loops.
- **Exact:** Doesn't require epsilon approximation

Disadvantages:

- **Inefficient:** algebraic derivative expressions could be significantly larger and contain many repeated expressions:
 - Why?



Expression Swell

Consider computing the derivative of an expression like:

$$f(x) = g(x)h(x)u(x)$$

If we apply the product rule, we get:

$$\frac{\partial}{\partial x} f(x) = h(x)u(x) \frac{\partial}{\partial x} g(x) + g(x)u(x) \frac{\partial}{\partial x} h(x) + g(x)h(x) \frac{\partial}{\partial x} u(x)$$

Notice that there is **significant redundancy**. This is especially the case with neural networks (from earlier):

$$\frac{\partial}{\partial W_i^{(1)}} y(\mathbf{x}, \mathbf{w}) = \boxed{y(\mathbf{x}, \mathbf{w})} (1 - \boxed{y(\mathbf{x}, \mathbf{w})}) W_j^{(2)} \boxed{\sigma(W^{(1)} \mathbf{x})} (1 - \boxed{\sigma(W^{(1)} \mathbf{x})}) x_i$$

$\sigma(W^{(2)} \sigma(W^{(1)} \mathbf{x}))$

Automatic Differentiation



Automatic Differentiation (Autodiff)

Automatically generate a **program** to compute the derivative **efficiently** from the “**forward**” **computation** of the the error.

- **Automatic:** derivative program is constructed by tracing the forward computation graph.
- **Accurate:** derivative is computed to machine precision.
- **Efficient:** reuse redundant computation

There are two approaches to Autodiff:

- **Forward Mode:** Best for $f: \mathbb{R} \rightarrow \mathbb{R}^D$
- **Backward Mode:** Best for $f: \mathbb{R}^D \rightarrow \mathbb{R}^1$ (deep learning)

Forward Tracing the Computation Graph

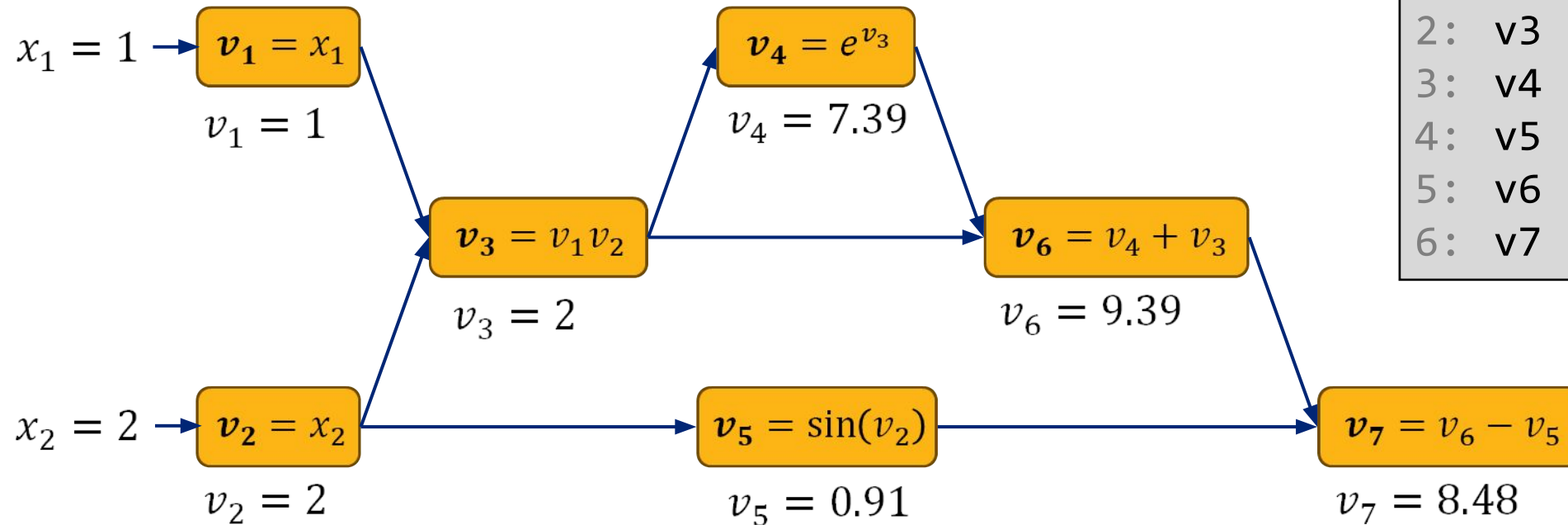


We construct the graph by evaluating the forward computation:

$$f(x_1, x_2) = x_1 x_2 + e^{(x_1 x_2)} - \sin(x_2)$$

Program:

```
0: v1 = x1
1: v2 = x2
2: v3 = v1 * v2
3: v4 = exp(v3)
4: v5 = sin(v2)
5: v6 = v4 + v3
6: v7 = v6 - v5
```

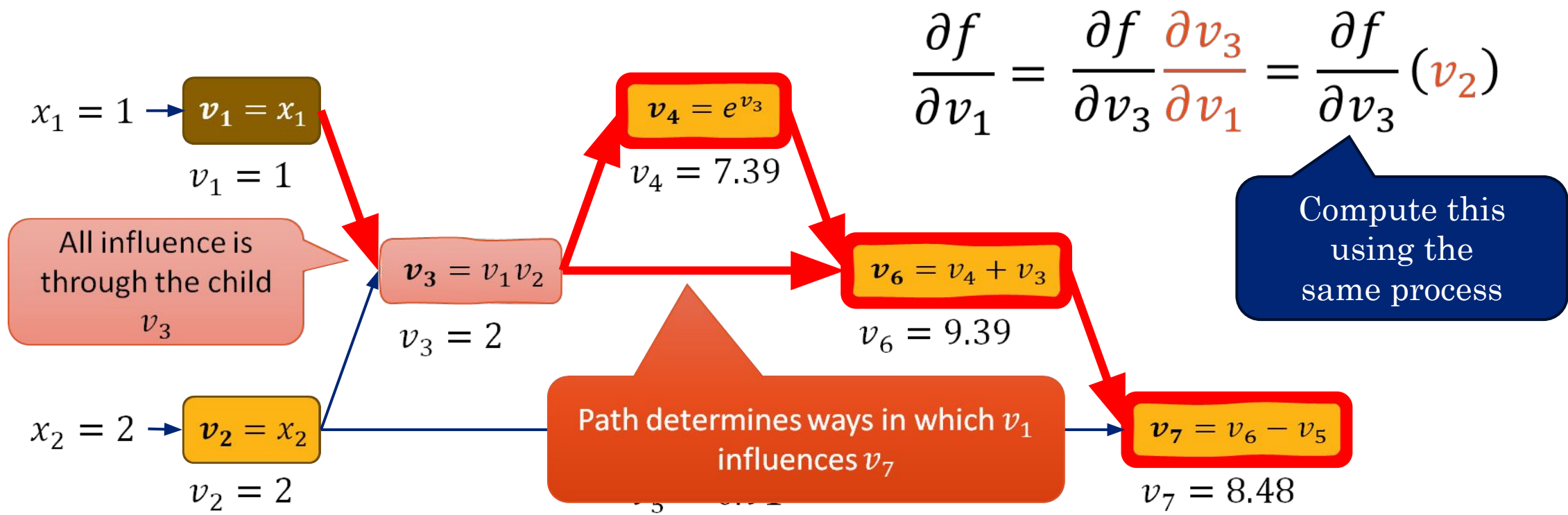


In this program we have broken each expression into a separate line.



Differentiating wrt v_1 Using the Chain Rule

Computing the derivative of $\frac{\partial f}{\partial v_1}$ by observing the **path** from v_1 to v_7 .



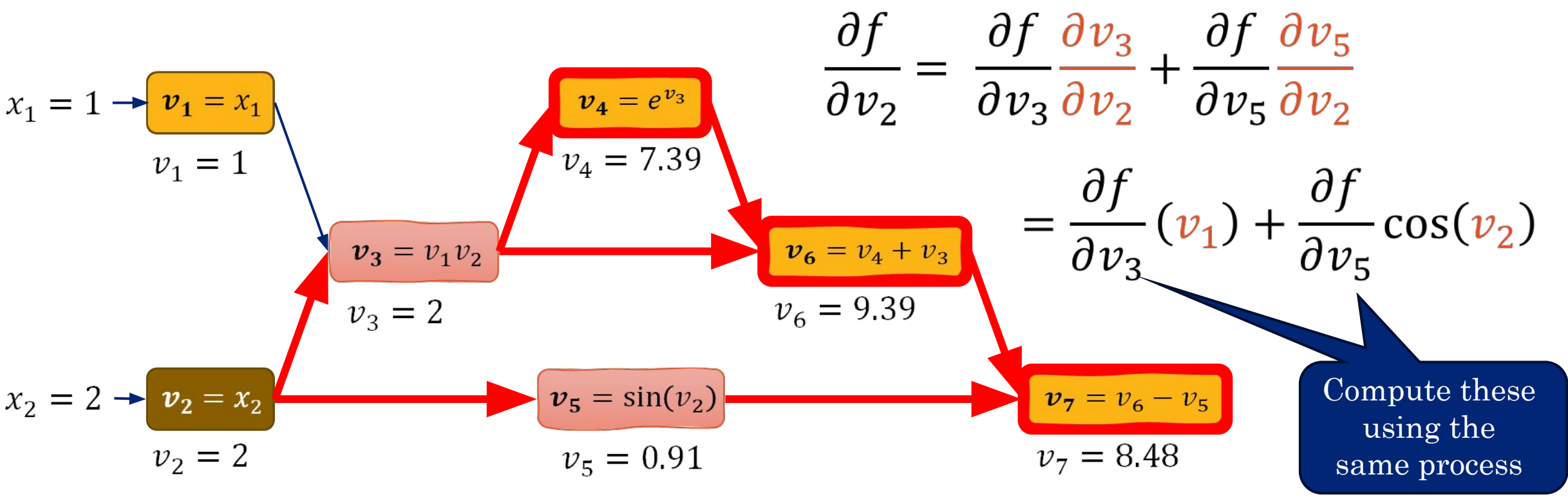
$$\frac{\partial f}{\partial v_1} = \frac{\partial f}{\partial v_3} \frac{\partial v_3}{\partial v_1} = \frac{\partial f}{\partial v_3} (v_2)$$

$$f(x_1, x_2) = x_1 x_2 + e^{(x_1 x_2)} - \sin(x_2)$$



Differentiating wrt v_2 Using the Chain Rule

Computing the derivative of $\frac{\partial f}{\partial v_2}$ by observing the paths from v_2 to v_7 .

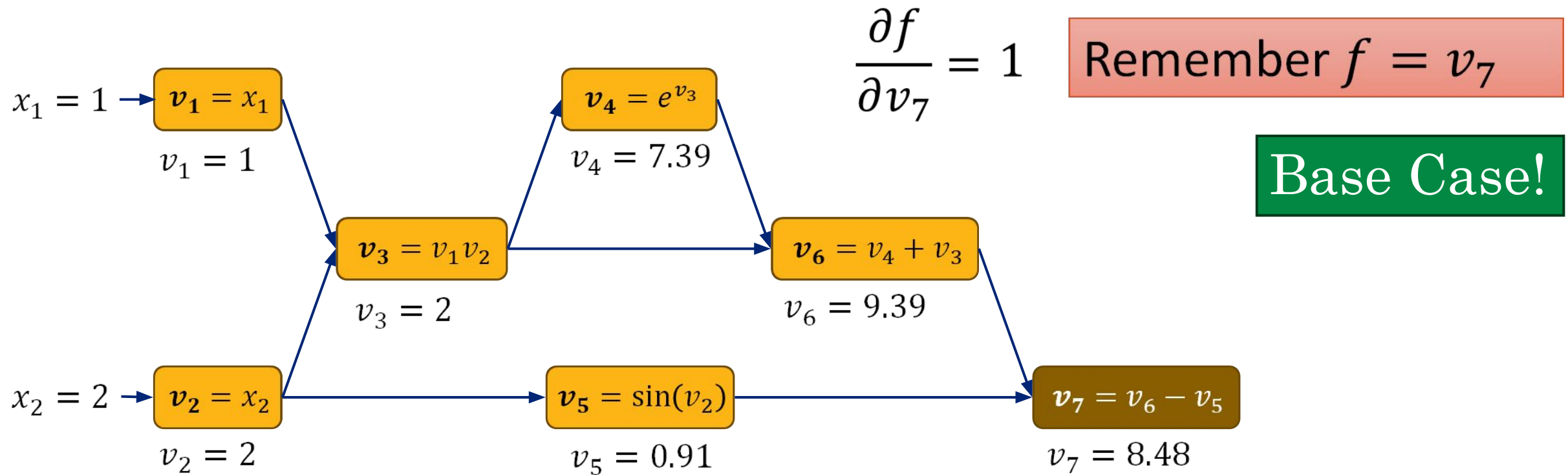


$$f(x_1, x_2) = x_1x_2 + e^{(x_1x_2)} - \sin(x_2)$$



Differentiating wrt v_7 Using the Chain Rule

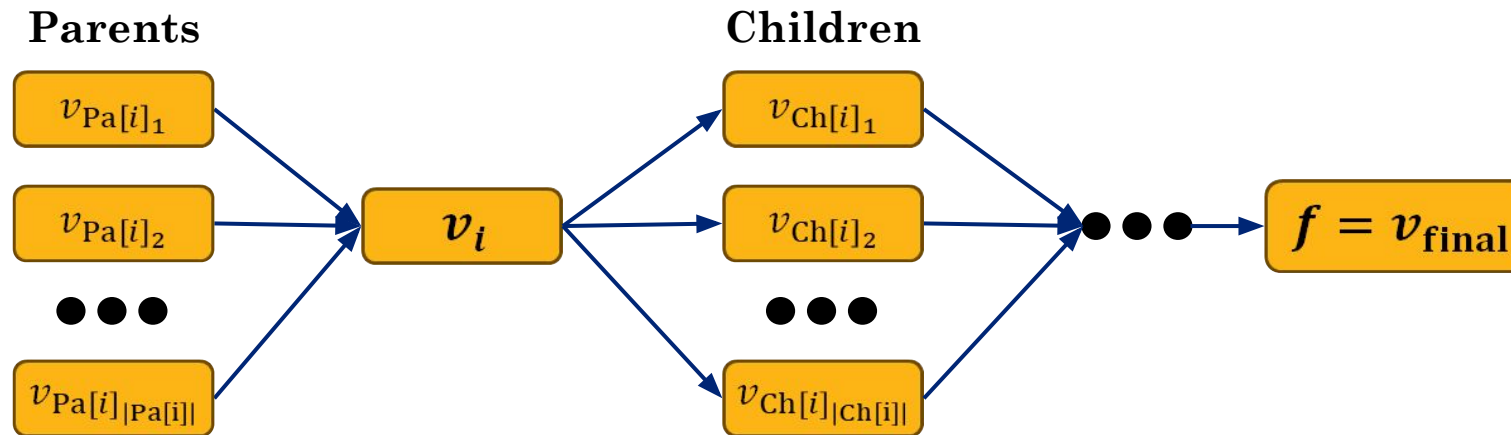
Computing the derivative of $\frac{\partial f}{\partial v_7}$ by observing the paths from v_7 to v_7 .





Backward Propagation (Recursion)

Consider the computation graph around v_i with a **single output** $f = v_{\text{final}}$



We introduce the **adjoint variable** a_i^f and use the **chain rule to obtain**:

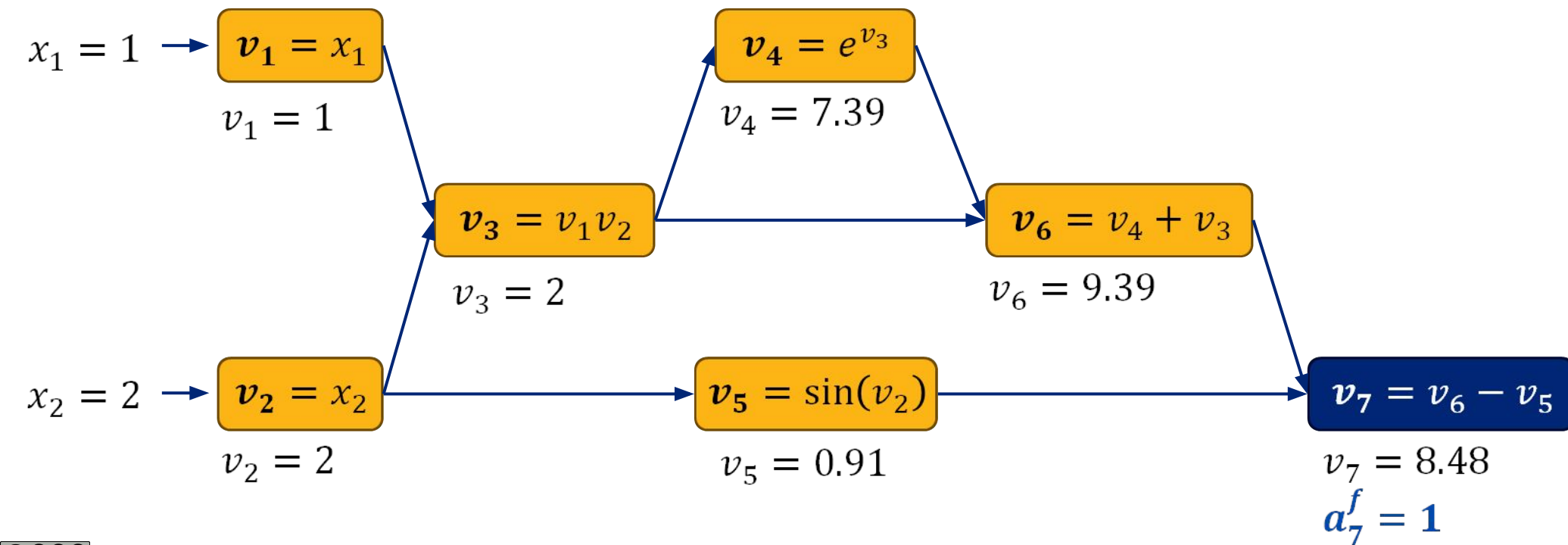
Recursion:
$$a_i^f = \frac{\partial f}{\partial v_i} = \sum_{j \in \text{Ch}[i]} \left(\frac{\partial f}{\partial v_j} \right) \left(\frac{\partial v_j}{\partial v_i} \right) = \sum_{j \in \text{Ch}[i]} a_j^f \left(\frac{\partial v_j}{\partial v_i} \right)$$

Where $\text{Ch}[i]$ is the children of vertex i in the computation graph.

Backward Autodiff Example

$$a_i^f = \sum_{j \in \text{Ch}[i]} a_j^f \left(\frac{\partial v_j}{\partial v_i} \right)$$

- ➔ 1. Initialize the final adjoint $a_f^f = \frac{\partial f}{\partial f} = 1$
2. Compute the adjoint for each variable **working backward**.



3940975

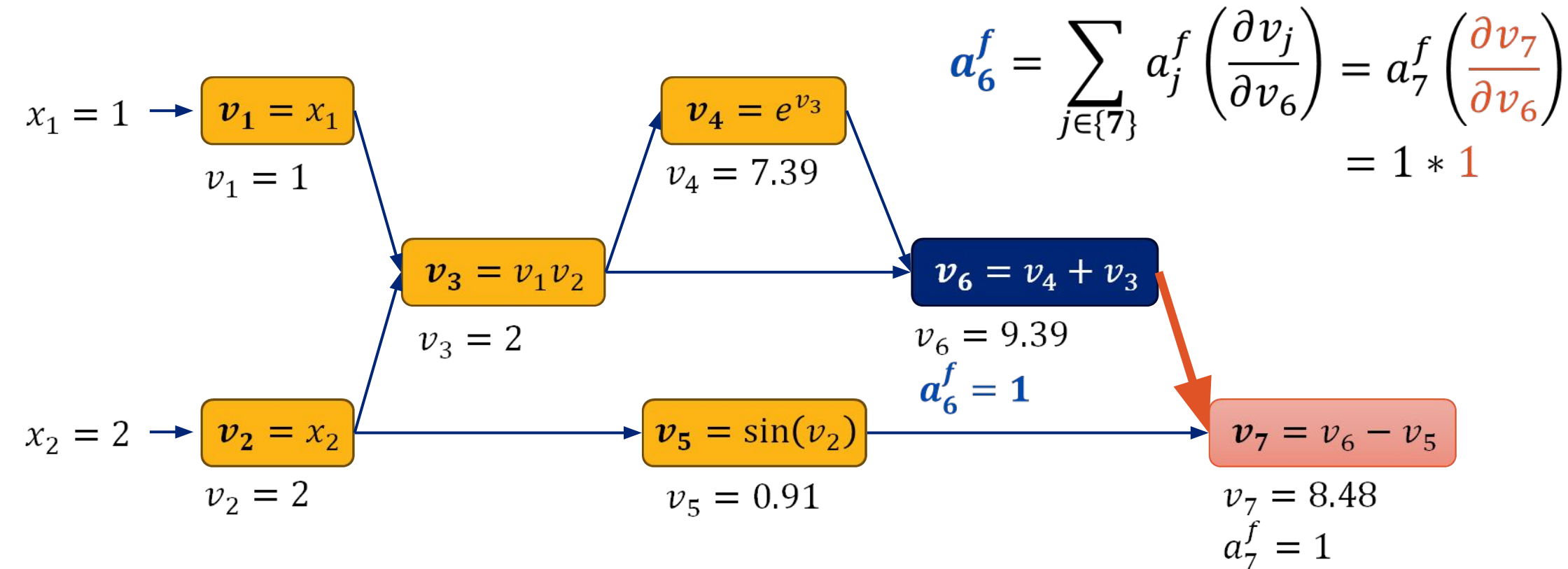


Backward Autodiff a_6^f

$$a_i^f = \sum_{j \in \text{Ch}[i]} a_j^f \left(\frac{\partial v_j}{\partial v_i} \right)$$

1. Initialize the final adjoint $a_f^f = \frac{\partial f}{\partial f} = 1$

➡ 2. Compute the adjoint for each variable **working backward**.



3940975



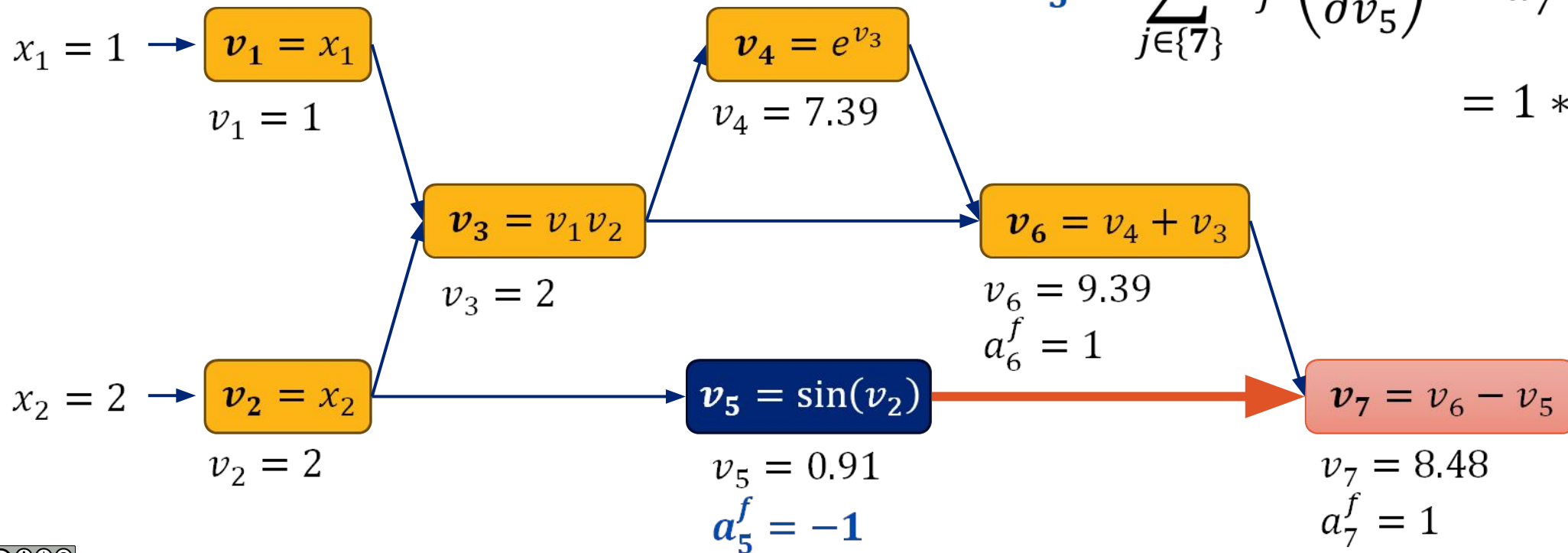
Backward Autodiff a_5^f

$$a_i^f = \sum_{j \in \text{Ch}[i]} a_j^f \left(\frac{\partial v_j}{\partial v_i} \right)$$

1. Initialize the final adjoint $a_f^f = \frac{\partial f}{\partial f} = 1$

➡ 2. Compute the adjoint for each variable **working backward**.

$$a_5^f = \sum_{j \in \{7\}} a_j^f \left(\frac{\partial v_j}{\partial v_5} \right) = a_7^f \left(\frac{\partial v_7}{\partial v_5} \right) = 1 * (-1)$$



3940975

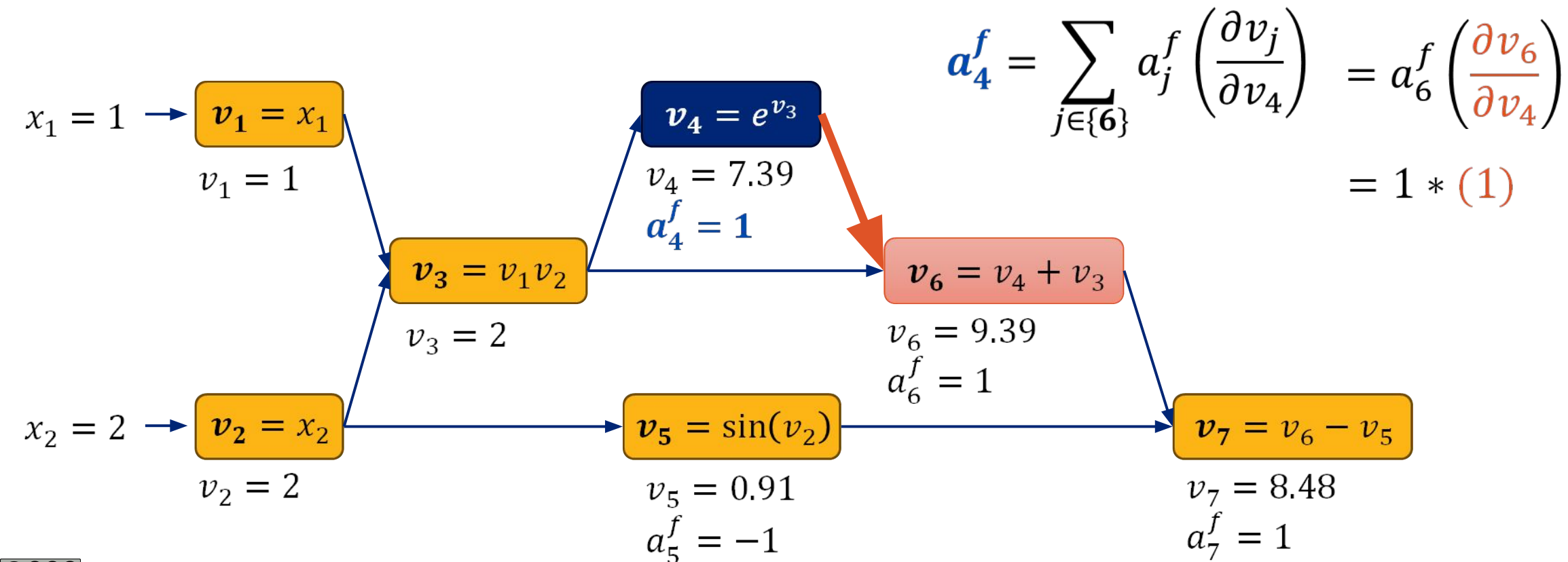


Backward Autodiff a_4^f

$$a_i^f = \sum_{j \in \text{Ch}[i]} a_j^f \left(\frac{\partial v_j}{\partial v_i} \right)$$

1. Initialize the final adjoint $a_f^f = \frac{\partial f}{\partial f} = 1$

➡ 2. Compute the adjoint for each variable **working backward**.



3940975

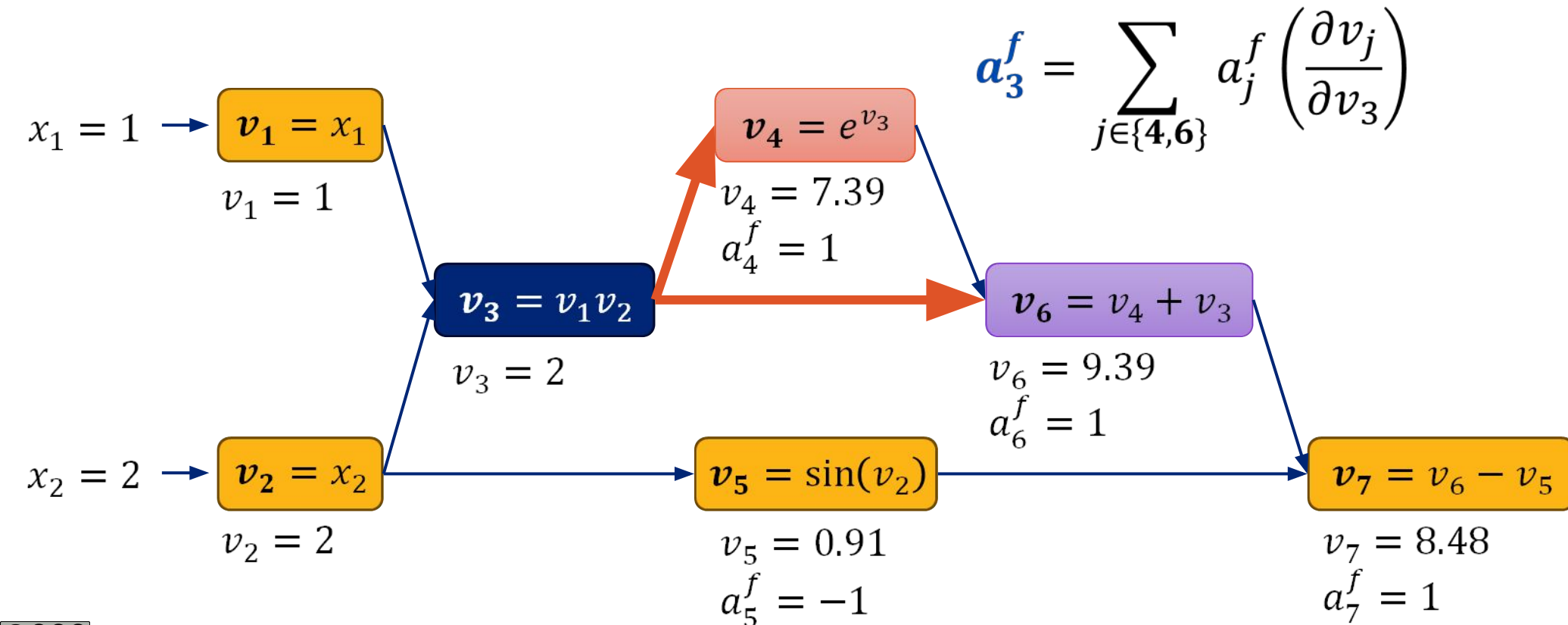


Backward Autodiff a_3^f

$$a_i^f = \sum_{j \in \text{Ch}[i]} a_j^f \left(\frac{\partial v_j}{\partial v_i} \right)$$

1. Initialize the final adjoint $a_f^f = \frac{\partial f}{\partial f} = 1$

➡ 2. Compute the adjoint for each variable **working backward**.



3940975





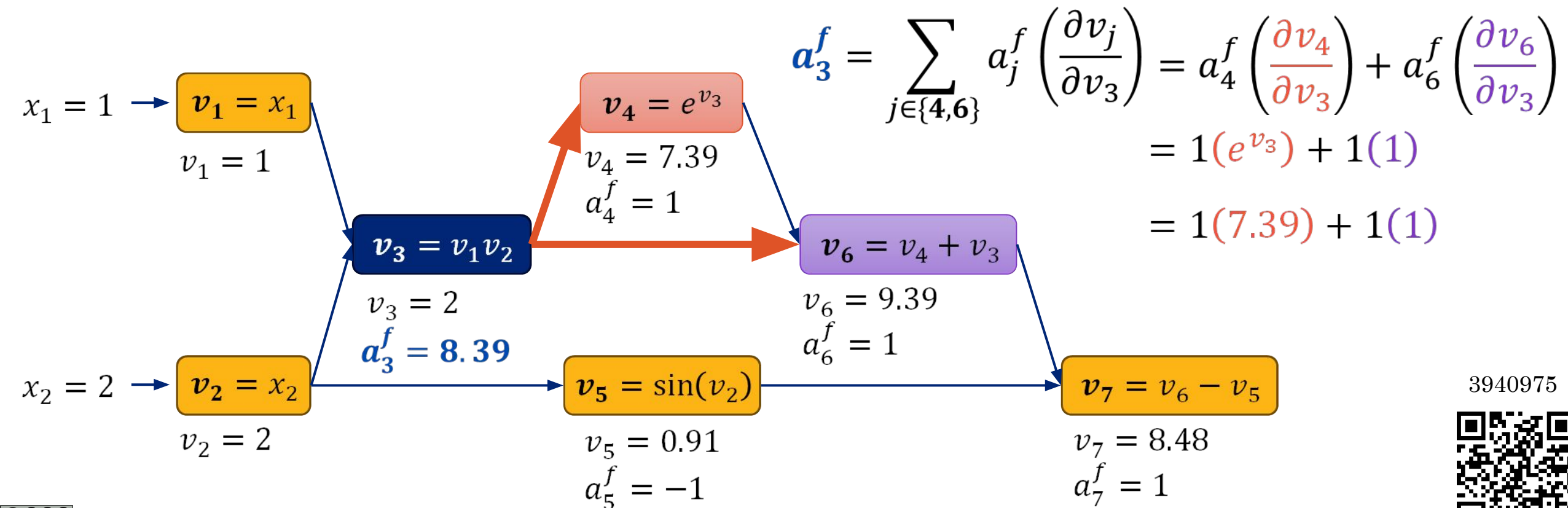
What is the value of the adjoint variable a_3 .

Backward Autodiff a_3^f

$$a_i^f = \sum_{j \in \text{Ch}[i]} a_j^f \left(\frac{\partial v_j}{\partial v_i} \right)$$

1. Initialize the final adjoint $a_f^f = \frac{\partial f}{\partial f} = 1$

➡ 2. Compute the adjoint for each variable **working backward**.



3940975

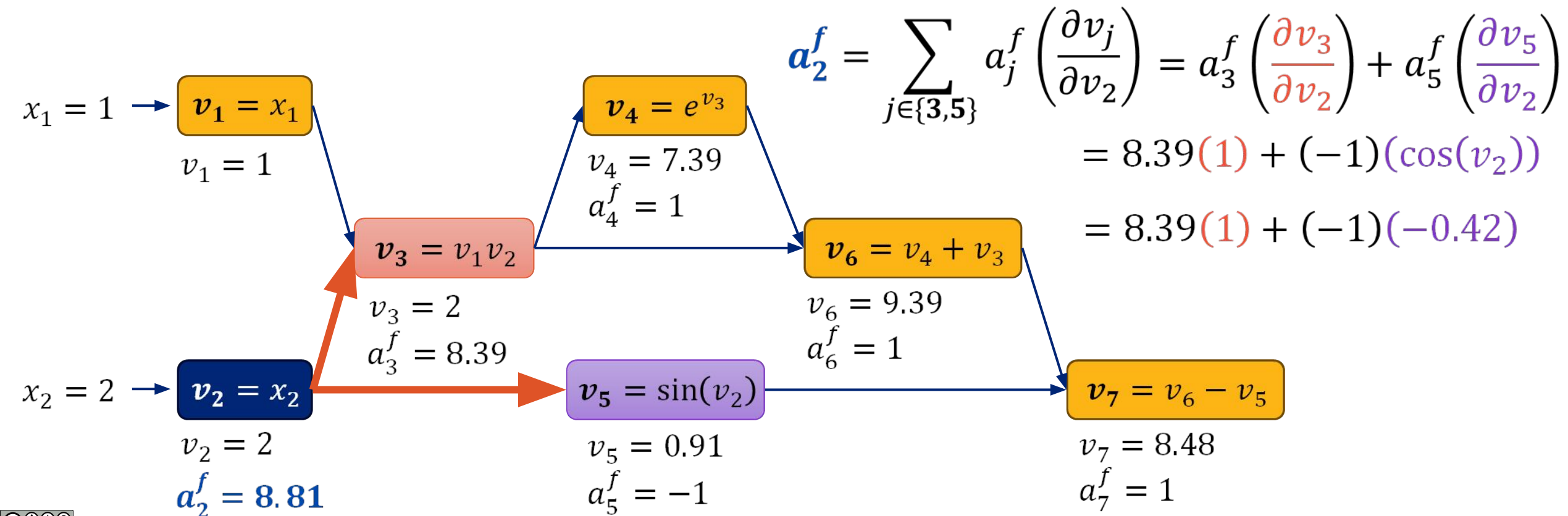


Backward Autodiff a_2^f

$$a_i^f = \sum_{j \in \text{Ch}[i]} a_j^f \left(\frac{\partial v_j}{\partial v_i} \right)$$

1. Initialize the final adjoint $a_f^f = \frac{\partial f}{\partial f} = 1$

➡ 2. Compute the adjoint for each variable **working backward**.

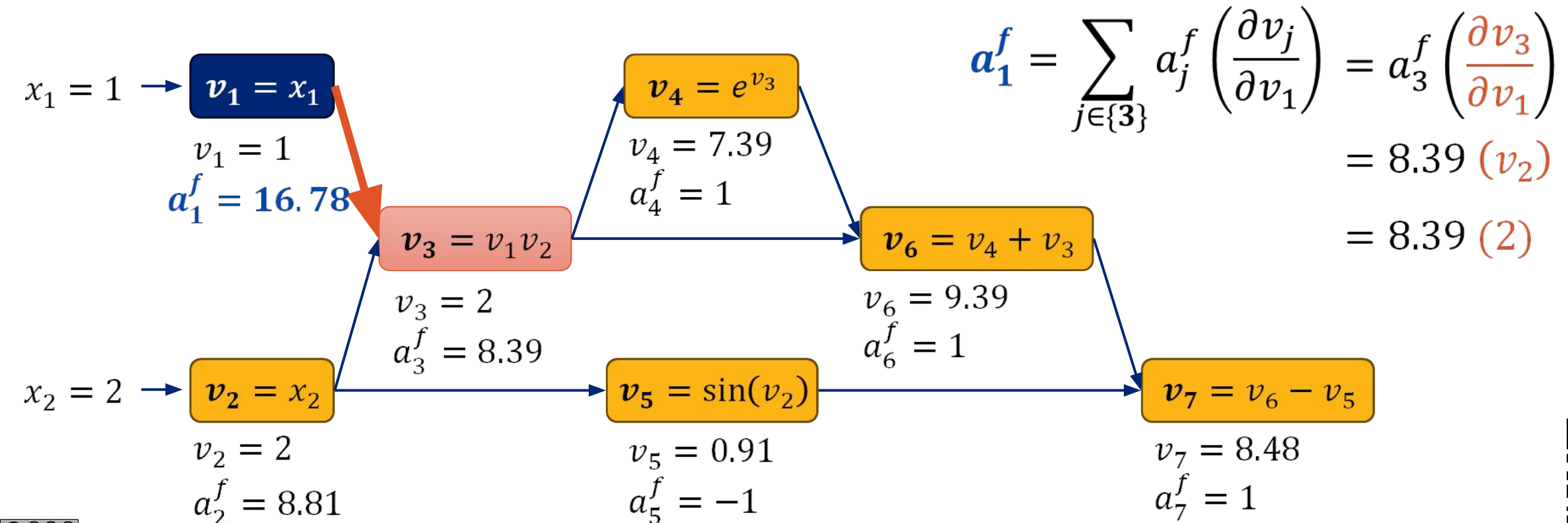


Backward Autodiff a_1^f

$$a_i^f = \sum_{j \in \text{Ch}[i]} a_j^f \left(\frac{\partial v_j}{\partial v_i} \right)$$

1. Initialize the final adjoint $a_f^f = \frac{\partial f}{\partial f} = 1$

➡ 2. Compute the adjoint for each variable **working backward**.



3940975

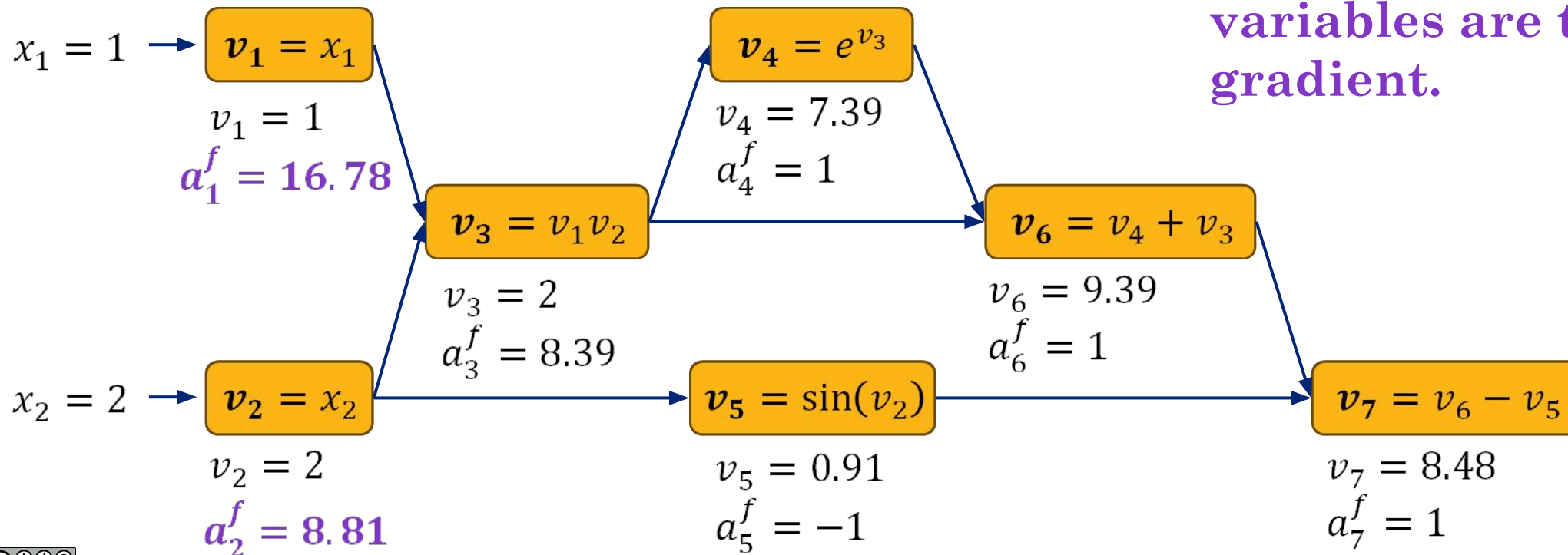


Obtaining The Gradient!

$$a_i^f = \sum_{j \in \text{Ch}[i]} a_j^f \left(\frac{\partial v_j}{\partial v_i} \right)$$

1. Initialize the final adjoint $a_f^f = \frac{\partial f}{\partial f} = 1$
2. Compute the adjoint for each variable **working backward**.

➡ 3. The final input adjoint variables are the gradient.

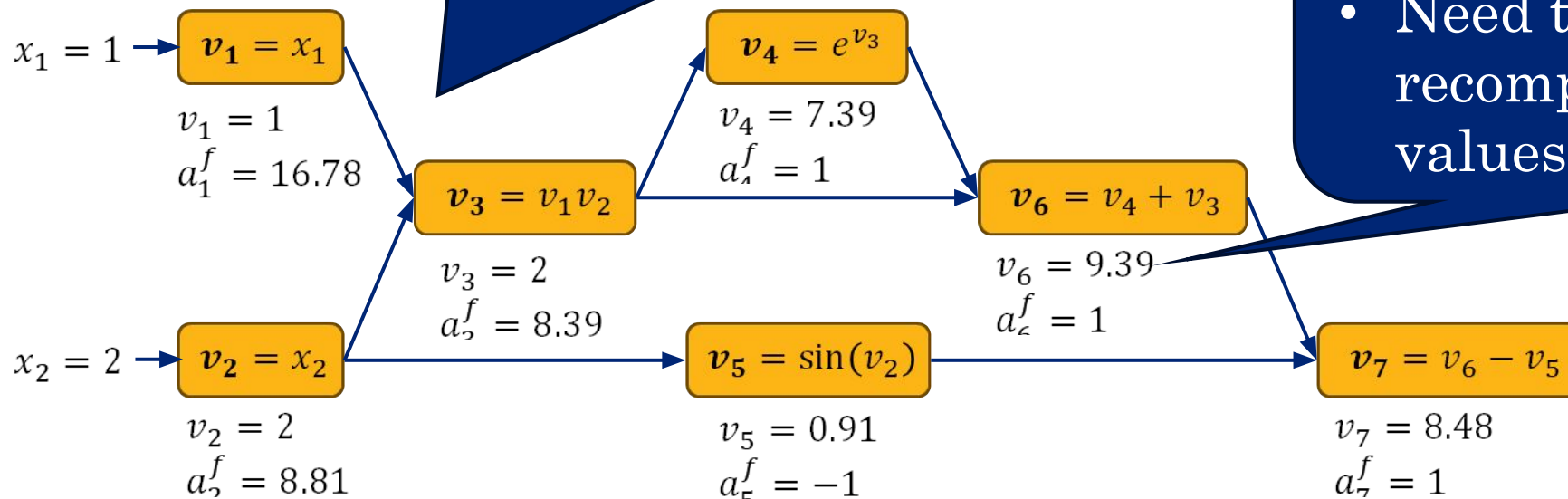


Cost Analysis



Backpropagation is efficient when there is a single output and many inputs.

Only need to maintain **single adjoint variable** at each step.



Often **reuse computation** from the forward step.

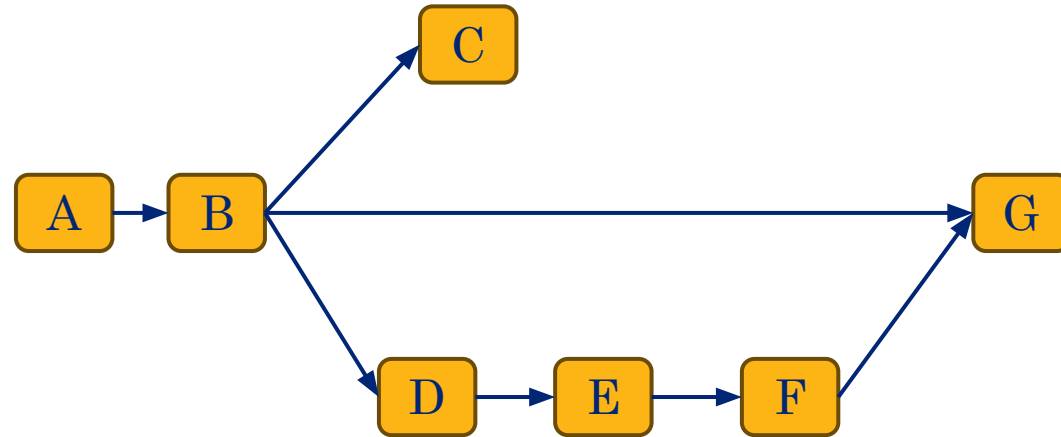
- Need to cache or recompute intermediate values

Need to run **backpropagation repeatedly** for multiple outputs.

Demo

You will do this in your homework.

- But my example code (and my example bug) may be helpful.



Hint:

The **adjoint at B** can't be computed until the **adjoint at D is computed** (topological ordering) but does not depend on the **adjoint at C** (an inactive path).





Additional Reading

- [Automatic Differentiation in Machine Learning: A Survey \(2018\)](#)
- [A Gentle Introduction to torch.autograd](#)

Lecture 15

PyTorch Continued & Backpropagation

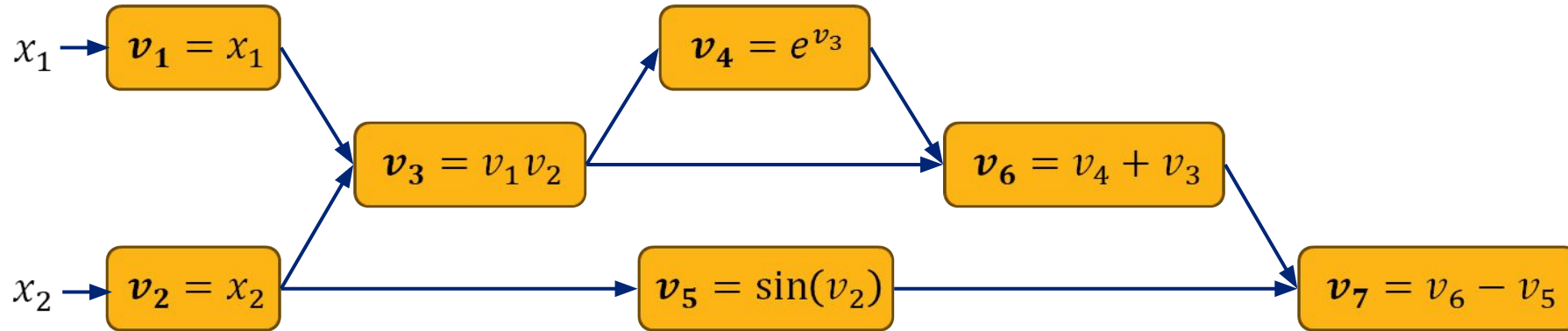
Reading: Chapter 8 in Bishop Deep Learning Textbook

Bonus Material

Forward Differentiation



Chain Rule on The Computation Graph



If we want to compute the derivative $\frac{\partial}{\partial x_2} f$ using the chain rule:

$$\frac{\partial v_7}{\partial x_2} = \frac{\partial v_7}{\partial v_6} \left(\frac{\partial v_6}{\partial x_2} \right) + \frac{\partial v_7}{\partial v_5} \left(\frac{\partial v_5}{\partial x_2} \right) = \left(\frac{\partial v_6}{\partial x_2} \right) - \left(\frac{\partial v_5}{\partial x_2} \right)$$

$$\frac{\partial v_3}{\partial x_2} = \frac{\partial v_3}{\partial v_1} \left(\frac{\partial v_1}{\partial x_2} \right) + \frac{\partial v_3}{\partial v_2} \left(\frac{\partial v_2}{\partial x_2} \right)$$

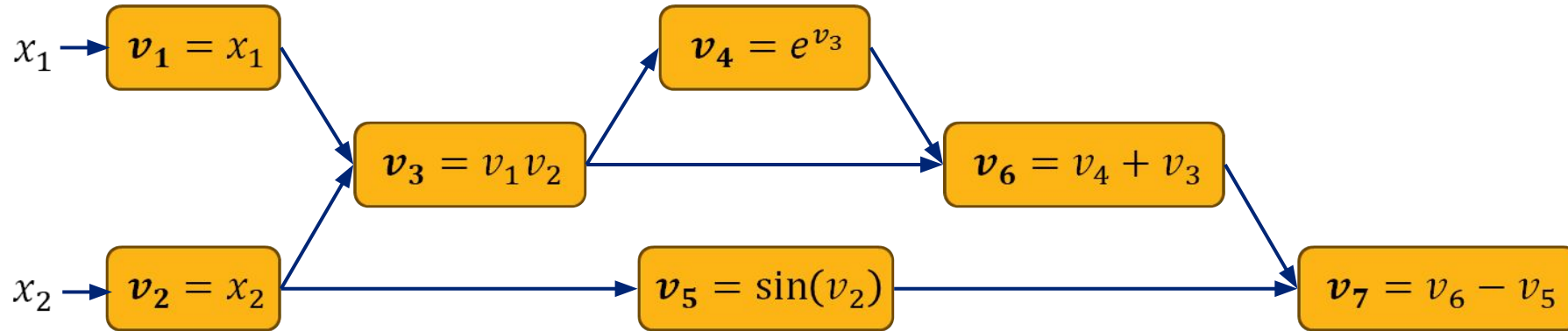
We can generalize these expressions with:

$$\frac{\partial v_i}{\partial x_2} = \sum_{j \in \mathbf{Pa}[i]} \left(\frac{\partial v_i}{\partial v_j} \right) \left(\frac{\partial v_j}{\partial x_2} \right)$$

where $\mathbf{Pa}[i]$ are the parents of i in the computation graph.



Chain Rule on The Computation Graph



If we define a new **tangent variable** representing the partial derivative $\partial_{x_k}^i = \frac{\partial v_i}{\partial x_k}$

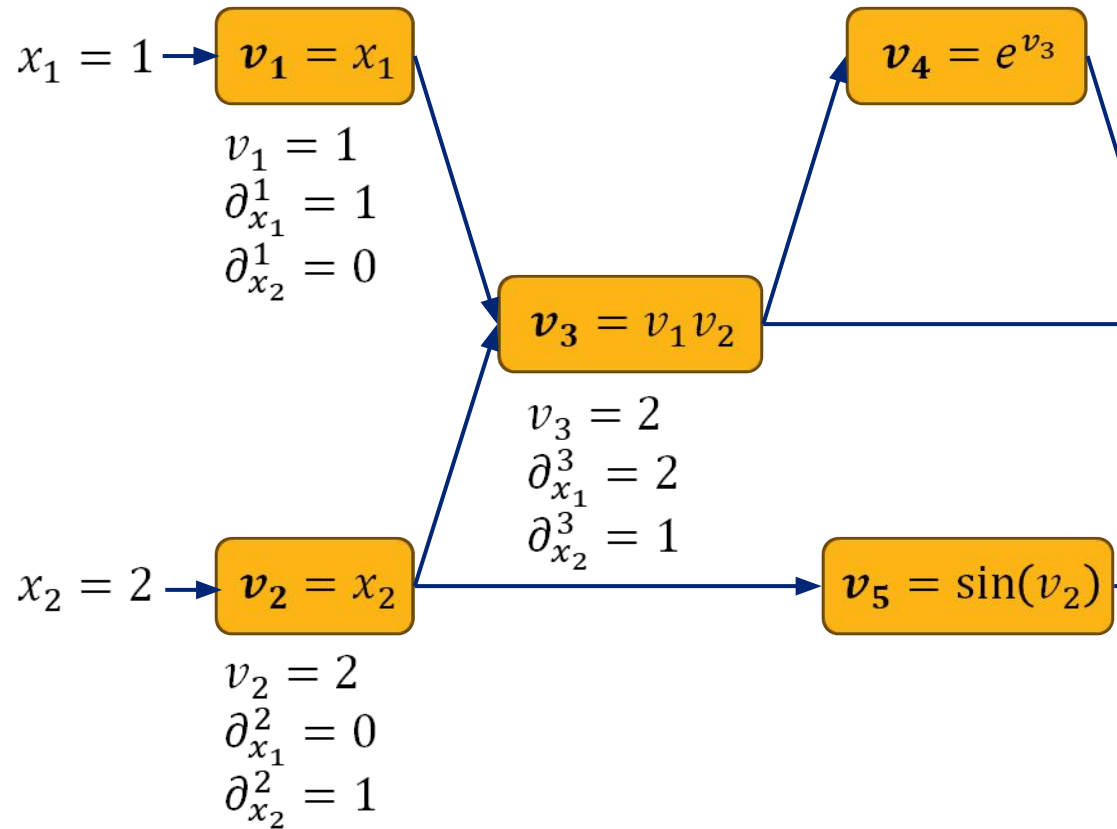
$$\partial_{x_k}^i = \frac{\partial v_i}{\partial x_k} = \sum_{j \in \text{Pa}[i]} \left(\frac{\partial v_i}{\partial v_j} \right) \left(\frac{\partial v_j}{\partial x_k} \right) = \sum_{j \in \text{Pa}[i]} \left(\frac{\partial v_i}{\partial v_j} \right) \partial_{x_k}^j$$

During the **forward computation** we can compute the value of v_i and $\partial_{x_k}^i$:

- Notice that v_i and $\partial_{x_k}^i$ only depend on their parents.
- Need to maintain separate $\partial_{x_k}^i$ for each variable k

Forward Autodiff Example

$$\partial_{x_k}^i = \sum_{j \in \text{Pa}[i]} \left(\frac{\partial v_i}{\partial v_j} \right) \partial_{x_k}^j$$



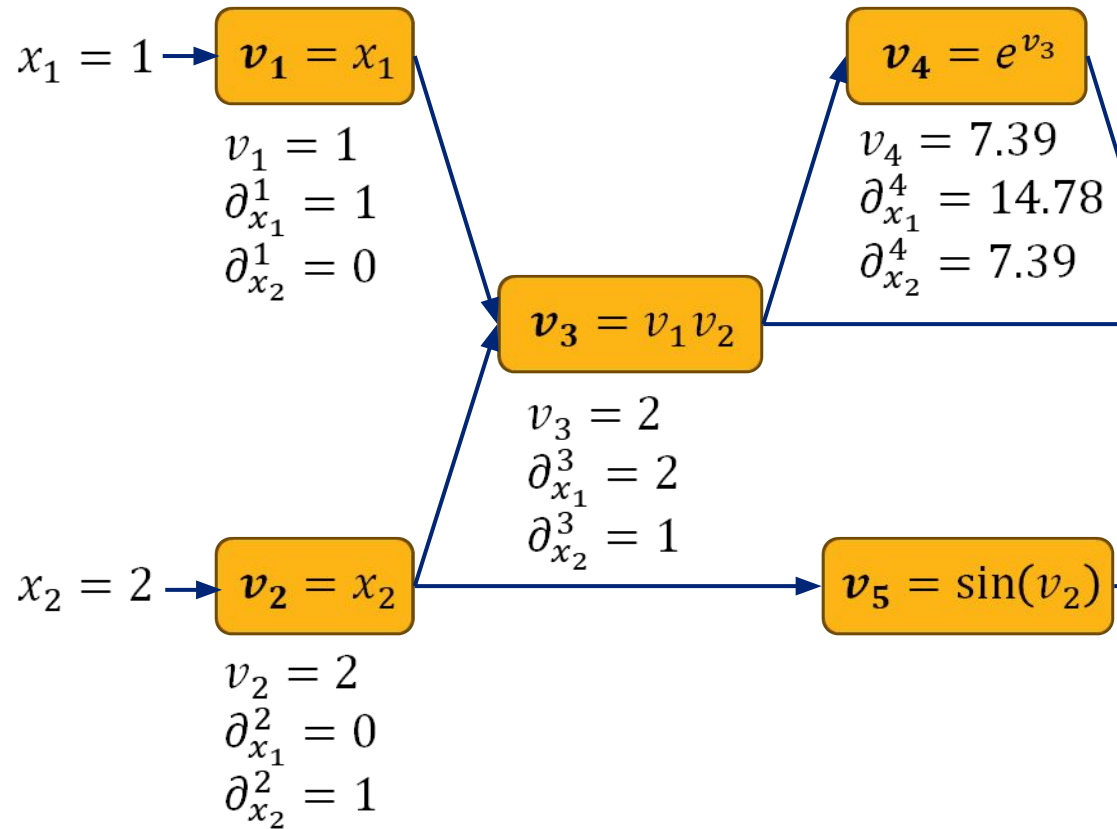
Derivation

$$\begin{aligned} \partial_{x_1}^3 &= \sum_{j \in \{1,2\}} \left(\frac{\partial v_3}{\partial v_j} \right) \partial_{x_1}^j = v_2 \partial_{x_1}^1 + v_1 \partial_{x_1}^2 \\ &= 2 * 1 + 1 * 0 \\ &= 2 \end{aligned}$$

$$\begin{aligned} \partial_{x_2}^3 &= \sum_{j \in \{1,2\}} \left(\frac{\partial v_3}{\partial v_j} \right) \partial_{x_2}^j = v_2 \partial_{x_2}^1 + v_1 \partial_{x_2}^2 \\ &= 2 * 0 + 1 * 1 \\ &= 1 \end{aligned}$$

Forward Autodiff Example

$$\partial_{x_k}^i = \sum_{j \in \text{Pa}[i]} \left(\frac{\partial v_i}{\partial v_j} \right) \partial_{x_k}^j$$



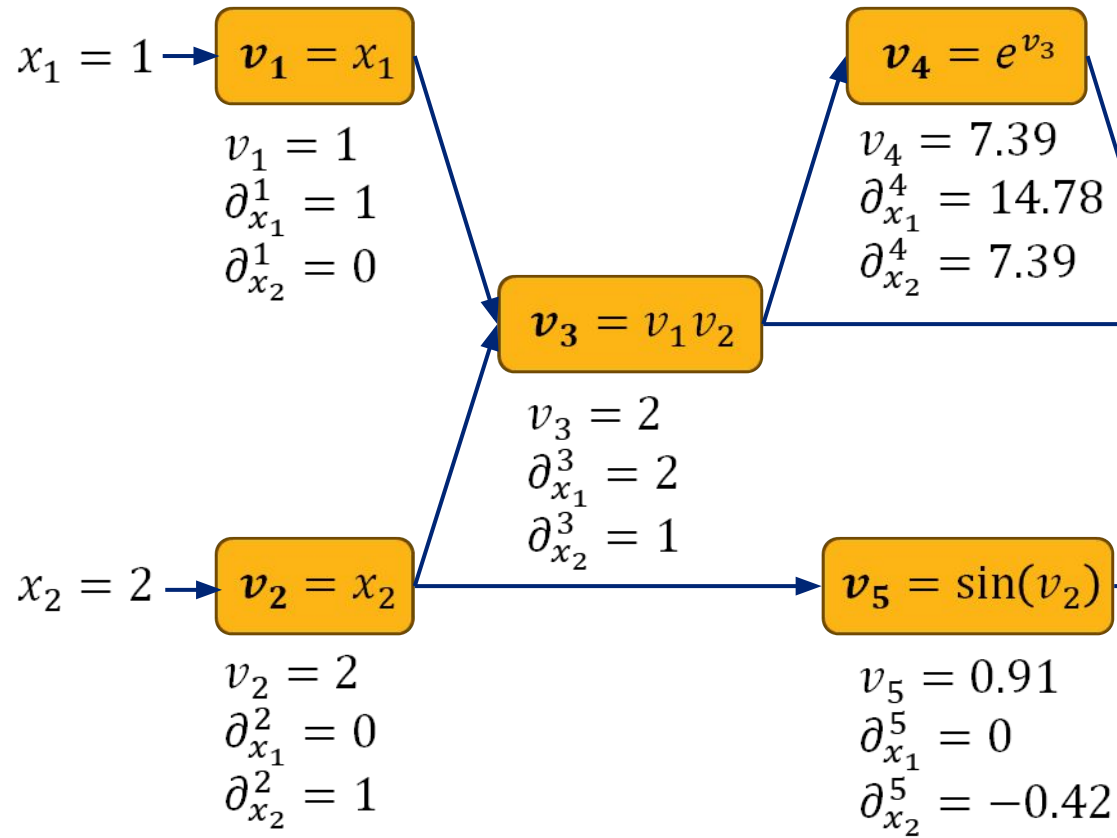
Derivation

$$\begin{aligned} \partial_{x_1}^4 &= \sum_{j \in \{3\}} \left(\frac{\partial v_4}{\partial v_j} \right) \partial_{x_1}^j = \left(\frac{\partial v_4}{\partial v_3} e^{v_3} \right) \partial_{x_1}^3 \\ &= (e^{v_3}) \partial_{x_1}^3 \\ &= (v_4) 2 = 14.78 \end{aligned}$$

$$\begin{aligned} \partial_{x_2}^4 &= \sum_{j \in \{3\}} \left(\frac{\partial v_4}{\partial v_j} \right) \partial_{x_2}^j = \left(\frac{\partial v_4}{\partial v_3} e^{v_3} \right) \partial_{x_2}^3 \\ &= (v_4) \partial_{x_2}^3 \\ &= (v_4) 1 = 7.39 \end{aligned}$$

Forward Autodiff Example

$$\partial_{x_k}^i = \sum_{j \in \text{Pa}[i]} \left(\frac{\partial v_i}{\partial v_j} \right) \partial_{x_k}^j$$



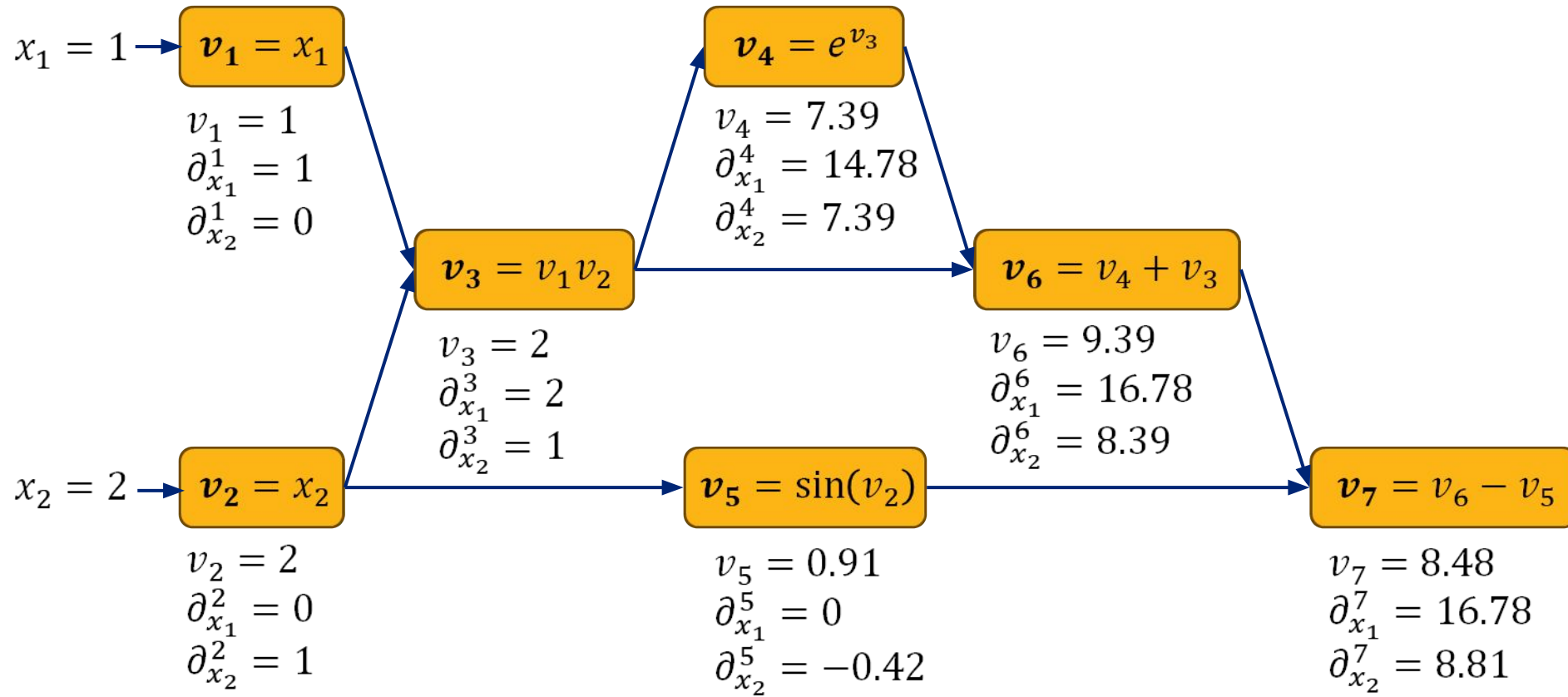
Derivation

$$\begin{aligned} \partial_{x_1}^5 &= \sum_{j \in \{2\}} \left(\frac{\partial v_5}{\partial v_j} \right) \partial_{x_1}^j = \left(\frac{\partial v_5}{\partial v_2} \sin(v_2) \right) \partial_{x_1}^2 \\ &= (\cos(v_2)) \partial_{x_1}^2 \\ &= (-0.42) 0 = 0 \end{aligned}$$

$$\begin{aligned} \partial_{x_2}^5 &= \sum_{j \in \{2\}} \left(\frac{\partial v_5}{\partial v_j} \right) \partial_{x_2}^j = \left(\frac{\partial v_5}{\partial v_2} \sin(v_2) \right) \partial_{x_2}^2 \\ &= (\cos(v_2)) \partial_{x_2}^2 \\ &= (-0.42) 1 = -0.42 \end{aligned}$$

Forward Autodiff Example

$$\partial_{x_k}^i = \sum_{j \in \text{Pa}[i]} \left(\frac{\partial v_i}{\partial v_j} \right) \partial_{x_k}^j$$

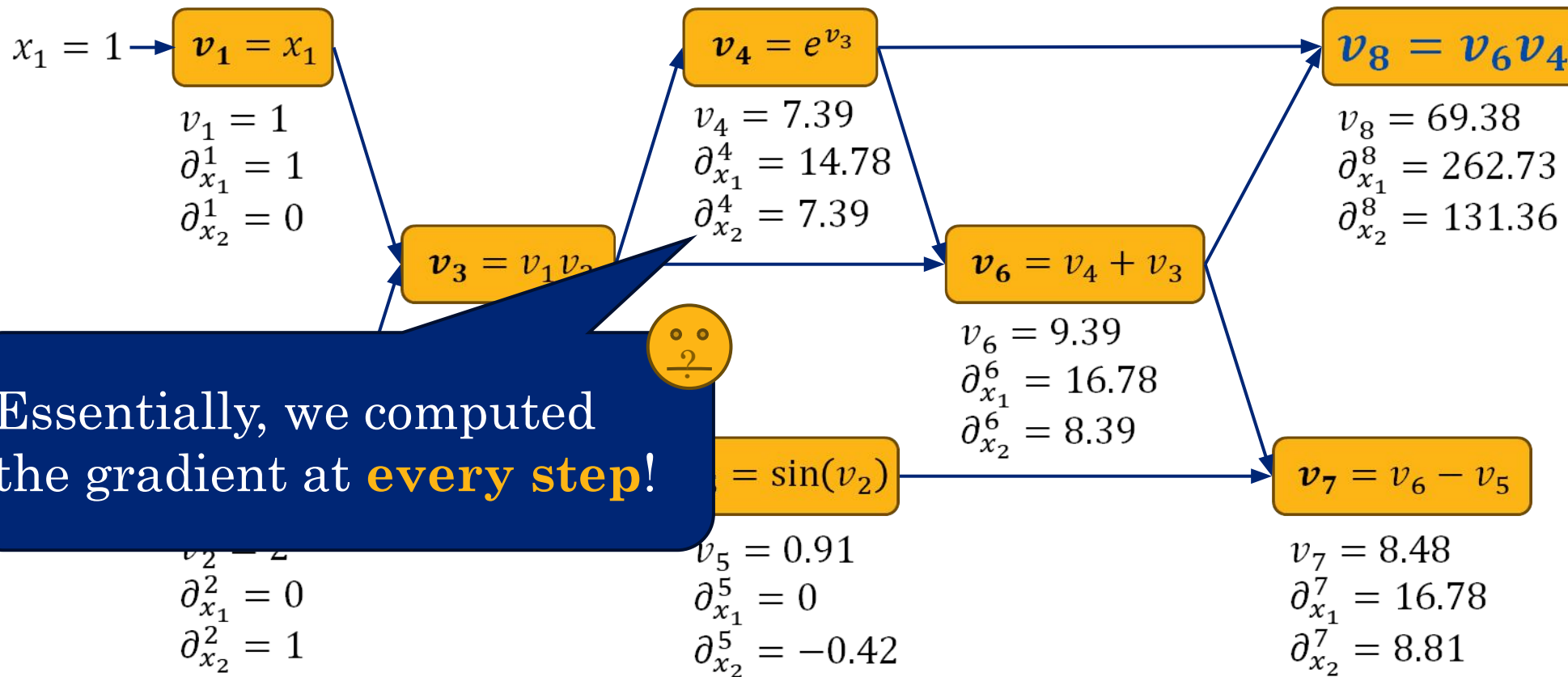


Support multiple outputs

$$v_7 = f_1(x_1, x_2) = x_1x_2 + e^{(x_1x_2)} - \sin(x_2)$$

$$v_8 = f_2(x_1, x_2) = (x_1x_2 + e^{(x_1x_2)})e^{(x_1x_2)}$$

Additional function output
can **reuse computation**.



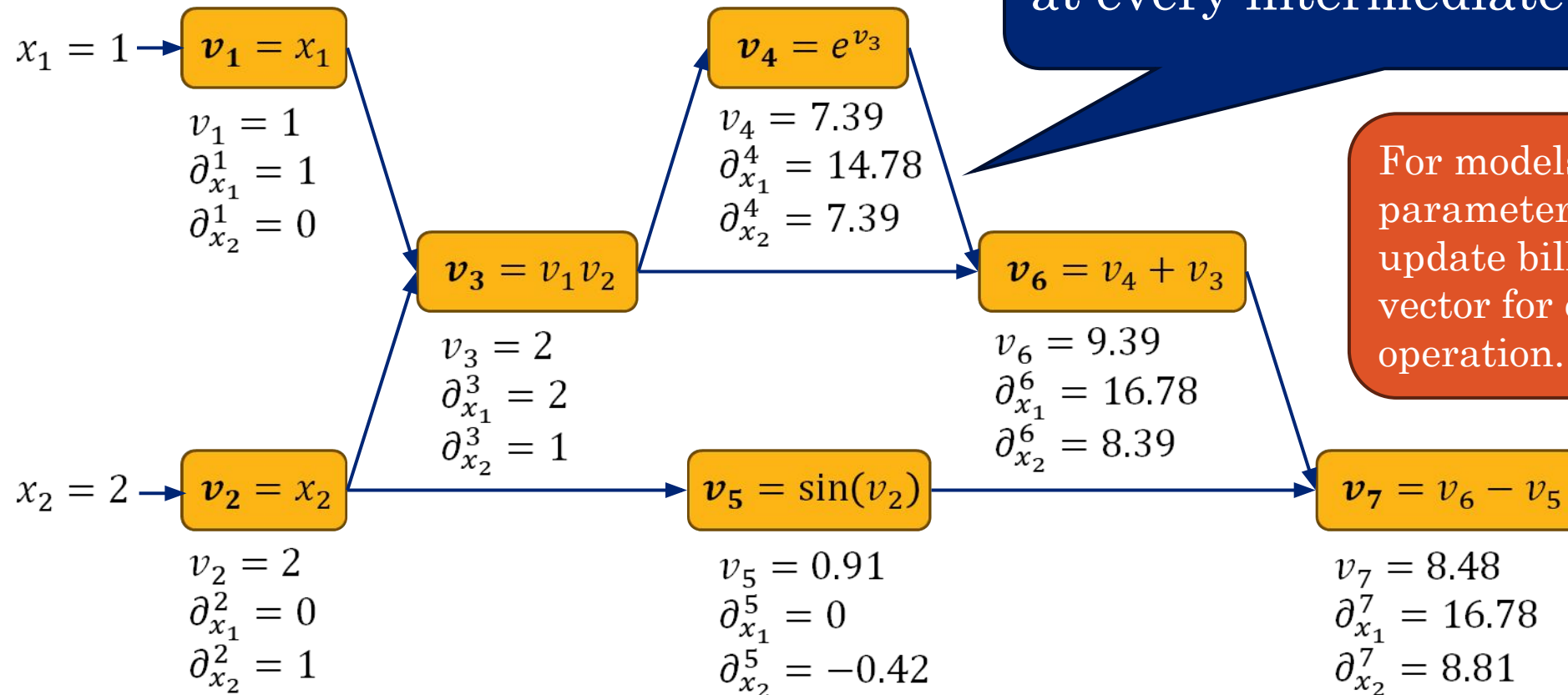
Expensive for Lots of Variables



3940975



Need to **maintain full gradient** at every intermediate step.



For models with billions of parameters, need to store and update billion-dimension vector for every arithmetic operation.



Adjoint vs Tangent Variables

We have now defined **tangent** and **adjoint variables** for **forward** and **backward** autodiff respectively:

Tangent Variables

$$\partial_{x_k}^i = \sum_{j \in \text{Pa}[i]} \left(\frac{\partial v_i}{\partial v_j} \right) \partial_{x_k}^j$$

Adjoint Variables

$$a_i^f = \sum_{j \in \text{Ch}[i]} a_j^f \left(\frac{\partial v_j}{\partial v_i} \right)$$

Notice how they build from one side of the graph to the other:

